Critique DPO

Zihe Liu, Peter Hayes

February 21, 2026

Abstract

This is *very much* a work-in-progress document, which reviews how generative verifiers can be implemented and simple experimental results. Much of this current version are disjointed concepts and messy; my current task is to tidy these reviews up and include the experimental results.

1 Introduction

As large language models (LLMs) become more capable, fine-tuning techniques to align with human intent are increasingly important. A key consideration for aligning these models is how to most effectively use human resources, or model resources in the case where LLMs themselves are used as oracles. Reinforcement learning from Human or AI preferences (RLHF/RLAIF) is the most prominent example of such a technique but is complex and often unstable. Direct Preference Optimization (DPO) has recently been proposed as a simpler and more stable alternative. In this note, we explore how to expand the DPO objective to make better use of feedback data from the human or AI oracles. Specifically, we incorporate the Chain-Of-Thought reasoning of a generative verifier (acting to verify and critique the validity of base models) in the DPO loss term for the base model.

Asking these oracles to provide reasoning for their binary preferences is common practice. In particular, for LLM-as-a-judge style workflows asking the model to provide reasoning (or so-called chain-of-thought) has been shown to actually improve the model predictions, or verifications in the case of generative verifiers. How can we leverage this rationale when fine-tuning our target model to improve performance in a more data-efficient manner?

2 Background: DPO

For a given prompt x, two completions are sampled from the policy $(y_w, y_l) \sim \pi_\theta(y|x)$ and an oracle judges which it prefers. We denote y_w as the preferred completion and y_l as the other. This process is repeated over N prompts resulting in the pairwise preference dataset $\mathcal{D} = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$.

2.1 Bradley-Terry Preference Model

We can model the judge preferences using the Bradley-Terry (BT) model, which expresses the probability that completion y_1 is preferred over y_2 under a latent reward function r^*:

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(r^*(x, y_1)\right)}{\exp\left(r^*(x, y_1)\right) + \exp\left(r^*(x, y_2)\right)} = \sigma(r^*(x, y_1) - r^*(x, y_2))$$

A key property is that BT depends only on the *difference* of rewards, not their absolute values.

2.2 Standard RLHF

The standard RLHF pipeline comprises 2 stages. First, we learn a reward model r_ϕ where we find the maximum likelihood estimate (MLE) of ϕ under our preference dataset $\mathcal{D}$ and BT model - termed reward modelling. Then, we use RL (PPO) to learn an optimal policy π_θ for this learned reward model.

2.2.1 Reward Modelling

Since r^* is unknown, we learn a parametric approximation r_ϕ. Under the BT model, the likelihood of the observed preference dataset is $\ell(\mathcal{D} \mid \phi) = \prod_{i=1}^{N} \sigma\Big(r_\phi(x^{(i)}, y_w^{(i)}) - r_\phi(x^{(i)}, y_l^{(i)})\Big)$. Taking the log and writing as an expectation, the MLE is obtained by minimising

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{1}$$

2.2.2 RL Fine-Tuning

The learned reward model is then used to fine-tune the policy. The aim is to maximise the KL-constrained objective

$$\max_{\pi_\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(y|x)}\left[r_\phi(x, y)\right] - \beta\, D_{\text{KL}}(\pi_\theta(y|x) \| \pi_{\text{ref}}(y|x)) \tag{2}$$

where π_{ref} is the reference policy (typically the SFT model), kept frozen. The KL term prevents the policy from deviating too far from the distribution on which the reward model is accurate, and $\beta > 0$ controls this trade-off. In practice this is optimised with PPO, requiring four models in memory: the policy, reference, reward model, and value function.

2.3 DPO

DPO [Rafailov et al., 2024] bypasses the reward modelling and RL steps entirely through a change of variables. The key insight is that there exists a bijection between reward functions and their corresponding optimal policies under the KL-constrained objective (2), and this bijection can be exploited to reparameterise the preference likelihood directly in terms of the policy. We follow the expositions by [Wolfe, 2025] and [Romero, 2024] in our presentation of the derivation below.

2.3.1 Optimal Policy

For *any* reward function r (not just the learned r_ϕ or the latent r^*), the KL-constrained objective (2) can be rewritten as a minimisation of a KL divergence between π and a target distribution defined by r, plus a constant. Specifically, letting $Z_r(x) = \sum_y \pi_{\text{ref}}(y|x) \exp\Big(\frac{1}{\beta} r(x, y)\Big)$ be a partition function that depends on x and r but is constant with respect to π, the objective becomes

$$\min_{\pi} \; \mathbb{E}_x \left[D_{\text{KL}}\left(\pi(y|x) \;\Big\|\; \frac{1}{Z_r(x)}\, \pi_{\text{ref}}(y|x)\, \exp\Big(\tfrac{1}{\beta}\, r(x, y)\Big)\right) - \log Z_r(x) \right]$$

By Gibbs' inequality ($D_{\text{KL}} \geq 0$, with equality iff the distributions are identical), the unique optimum is

$$\pi_r(y|x) = \frac{1}{Z_r(x)}\, \pi_{\text{ref}}(y|x)\, \exp\left(\frac{1}{\beta}\, r(x, y)\right) \tag{3}$$

This defines a bijection: every reward r maps to a unique optimal policy π_r, and conversely every policy π_r defines a unique reward[1] r for which it is optimal. Using the latent reward r^* gives the optimal policy $\pi^* \equiv \pi_{r^*}$ for our latent rewards.

2.3.2 Reparameterisation

Rearranging (3) for the reward expresses any reward purely in terms of its corresponding optimal policy and a prompt-dependent partition function:

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z_r(x)$$

[1] We really mean unique equivalence class of rewards and its optimal policy; see Lemma 1 and Lemma 2 in [Rafailov et al., 2024].

Substituting the latent reward $r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z_{r^*}(x)$ into the BT model, the $\beta \log Z_{r^*}(x)$ terms cancel because BT depends only on reward *differences* and $Z_{r^*}(x)$ is independent of y:

$$p^*(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

The preference model now has no dependence on a reward model — it is expressed entirely in terms of policies.

2.3.3 DPO Loss

We replace the fixed π^* with a learnable π_θ and maximise the likelihood of the observed preferences, conditioned on our policy parameters θ. Defining the *implicit reward* $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$, the DPO loss is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)\right)\right] \tag{4}$$

This is identical in form to the reward model loss (1), but conditioned on the policy parameters θ instead of the reward model parameters ϕ. Traditionally, RLHF performs MLE on $\mathcal{D}$ conditioned on ϕ to learn a reward model, then uses RL to find the optimal policy for that reward. DPO instead performs MLE on $\mathcal{D}$ conditioned directly on θ. Because of the reward–policy bijection, improving π_θ to better fit the preference data simultaneously yields an implicit reward $\hat{r}_\theta$ for which π_θ is guaranteed to be optimal. This eliminates both the reward modelling step and the RL step, requiring only two models in memory (the policy π_θ and the frozen reference π_{ref}).

In this work we suggest simple extensions to this objective.

3 Literature Review

We review research incorporating reasoning and critique data into alignment and fine-tuning processes, building toward the critique-augmented preference optimisation approach of this work. The review is organised around a central question: *what kind of signal is being used, and for which model?* The narrative arc moves from binary AI feedback (Section 3.1), through the value of chain-of-thought data as a training signal (Section 3.2), to methods that enrich preference optimisation with reasoning annotations for the policy model (Section 3.3), methods that improve verifiers and reward models by having them reason (Section 3.4), and finally to the closest work — methods that bring verifier critique directly into the base model's training objective (Section 3.5).

3.1 RLAIF: AI Feedback for Alignment

[Bai et al., 2022] first introduce RL from AI Feedback (RLAIF) to fine-tune models to be helpful and harmless. They use human labels for helpfulness, but LLM-generated labels for ranking harmlessness responses. They experiment with Chain-Of-Thought (CoT) prompting to generate the preferences to rank harmlessness, and find that RL-CAI with CoT seems slightly less helpful but slightly more harmless compared to without CoT. [Lee et al., 2024] compare RLAIF with RLHF on summarization, helpful dialogue generation, and harmless dialogue generation. Experiments show RLAIF achieves comparable, or even better, performance than RLHF even when the labeller model is the same as the base policy model. Again, experiments show CoT created minimal gains in the quality of the post-RLAIF model. [Guo et al., 2024] use LLM-as-a-judge to rank preferences for DPO training online. [Li et al., 2024] further instruct the LLM labeller to use explicit reasoning when generating preference data, combining CoT prompting with RLAIF to improve both helpfulness and harmlessness.

A shared limitation of these approaches is their reliance on a binary or scalar preference label as the sole training signal. The reasoning the oracle uses to reach that label is discarded after labelling, representing an information bottleneck that the methods in the remainder of this review seek to address.

3.2 Chain-of-Thought as a Training Signal

The earliest work to use chain-of-thought *as a training target* — rather than merely as a prompting strategy — established that explicit reasoning traces are a qualitatively richer signal than outcome labels alone.

3.2.1 Self-Taught Reasoning: STaR and V-STaR

STaR [Zelikman et al., 2022] first used CoT data for better performance. They used an iterative approach for improving LLM performance through rationale-based fine-tuning. Given a pretrained model M and a dataset $D = \{(x_i, y_i)\}_{i=1}^{n}$, we start with a small prompt set $\mathcal{P} = \{(x_i^p, r_i^p, y_i^p)\}_{i=1}^{P}$ where $P \ll D$ that includes examples with their rationales r and answers y. For each example in D, we concatenate the prompt set to form $x_i = (x_1^p, r_1^p, y_1^p, \ldots, x_P^p, r_P^p, y_P^p, x_i)$. The model generates rationales $\hat{r}_i$ and answers $\hat{y}_i$. We filter to keep only rationales that lead to correct answers ($\hat{y}_i = y_i$), fine-tune M on this filtered dataset, and iterate this process. Importantly, each iteration starts from the original pre-trained model M to prevent overfitting. The process continues until performance convergence.

Similarly, V-STaR [Hosseini et al., 2024] use an iterative approach to train a generator G^t and verifier V^t. For each problem in the training data, we sample k completions from the generator: $\{\hat{y}_i \sim G(\cdot|\mathbf{x}_i) : i \in \mathcal{D}_{query}\}$. The generated solutions are labeled for correctness (z) using ground truth answers or test cases. We use only correct generated solutions ($z = 1$) to augment the generator training data $\mathcal{D}_{GEN}$. Both correct and incorrect solutions are added to verifier data $\mathcal{D}_{VER}$ with their correctness labels as $(\mathbf{x}_i, \hat{y}_{ij}, z_{ij})$, enabling the verifier to learn from generator mistakes. At iteration t, generator G^t is obtained by fine-tuning the pretrained model on the augmented $\mathcal{D}_{GEN}$. We sample solutions again from this generator G^t. The verifier V^t is obtained by using $\mathcal{D}_{VER}$ with DPO to fine-tune a base model G_{SFT}, with preference datasets for correct and incorrect answers. This process repeats for up to T iterations to iteratively augment $\mathcal{D}_{GEN}$ and $\mathcal{D}_{VER}$, V^t and G^t.

3.2.2 The Role of CoT in DPO Training

[H. Liu et al., 2025] provide a systematic empirical investigation of why DPO succeeds on reasoning tasks. In the text-to-SQL domain — where preference datasets traditionally contain only final SQL queries without intermediate reasoning steps — they find that DPO *degrades* performance in the majority of configurations across ten LLMs. The core diagnosis is that without CoT annotations in the training data, DPO provides insufficient credit assignment and is vulnerable to reward hacking: models latch onto superficial correlates of preference rather than reasoning quality. Augmenting SQL datasets with synthetically generated chain-of-thought solutions achieves consistent DPO improvements in this domain for the first time. The analysis identifies three mechanisms by which CoT data benefits DPO: it mitigates reward hacking, improves discriminative calibration, and enhances training scalability. This finding directly motivates the Critique DPO approach: if the model's own CoT improves DPO, an external verifier's critique CoT — which provides a richer, oracle-level perspective on *why* one response is preferred — should be even more informative.

3.3 Preference Optimization with Reasoning Data

These methods enrich the DPO training signal with explicit reasoning annotations, while continuing to train the base policy model that generates answers.

3.3.1 Self-Rewarding Language Models

[W. Yuan et al., 2024] use language models to generate candidate responses, provide their own rewards to these responses, filter for good responses, and do iterative fine-tuning on these good responses. As the model can both improve its generation ability and act as its own reward model through the same generation mechanism, the reward model itself can improve through these iterations, deviating from standard practices where the reward model is fixed.

Self-Instruction Creation We generate a new prompt x_i every iteration using few-shot prompting, sampling prompts from original seed data. We then generate N diverse candidate responses $\{y_i^1, ..., y_i^N\}$ for the given prompt x_i from our model using sampling. Finally, we use the LLM-as-a-Judge ability of our *same model* to evaluate its own candidate responses with scores $r_i^n \in [0, 5]$.

AI Feedback Training After performing the self-instruction creation procedure, we can augment the seed data with additional examples for training, which we refer to as AI Feedback Training (AIFT) data. To do this, we construct preference pairs, which are training data of the form (instruction prompt x_i,

winning response y_i^w, losing response y_i^l). To form the winning and losing pair we take the highest and lowest scoring responses from the N evaluated candidate responses, discarding the pair if their scores are the same. These pairs can be used for training with a preference tuning algorithm DPO.

Iterative Training Our overall procedure trains a series of models $M_1, ..., M_T$ where each successive model t uses augmented training data created by the $t-1^{th}$ model. We thus define AIFT(M_t) to mean AI Feedback Training data created using model M_t. This iterative training resembles the procedure used in Pairwise Cringe Optimization and specifically is termed Iterative DPO, introduced in [Xu et al., 2024]; however, an external fixed reward model was used in that work.

3.3.2 Iterative Reasoning Preference Optimization

[Pang et al., 2024] optimize the preference between competing generated Chain-of-Thought (CoT) candidates by optimizing for winning vs. losing reasoning steps. They use a modified form of the DPO loss with an additional NLL term on the reasoning data. There is a large improvement from 55.6% to 81.6% on GSM8K, and other math benchmarks like MATH and ARC-Challenge.

The authors first construct a dataset of preference pairs, with each pair consisting of a winning (correct) answer and losing (incorrect) answer, along with the LLM's reasoning chain c_i: $D^{\text{pairs}} = \{(c_i^{w_k}, y_i^{w_k}), (c_i^{l_k}, y_i^{l_k})\}$ Note that due to the task of math reasoning, the reward is binary, either 1 or 0 depending on whether the final answer y_i was correct or not. Given the preference pairs, they then train a new model M_θ that will become the next model M_{i+1}. A sum of the DPO loss and NLL loss over the reasoning chain is used

$$\begin{aligned}\mathcal{L}_{\text{DPO+NLL}} &= \mathcal{L}_{\text{DPO}}(c_i^w, y_i^w, c_i^l, y_i^l|x_i) + \alpha\mathcal{L}_{\text{NLL}}(c_i^w, y_i^w|x_i) \\ &= -\log\sigma\left(\beta\log\frac{M_\theta(c_i^w, y_i^w|x_i)}{M_i(c_i^w, y_i^w|x_i)} - \beta\log\frac{M_\theta(c_i^l, y_i^l|x_i)}{M_i(c_i^l, y_i^l|x_i)}\right) - \alpha\frac{\log M_\theta(c_i^w, y_i^w|x_i)}{|c_i^w| + |y_i^w|}\end{aligned} \tag{5}$$

Here $M(x)$ denotes the probability of sequence x under the model M, and σ is the sigmoid function. We use the previous iteration's model M_i as the reference model in the denominator of the DPO term. Note that the NLL term is normalized by the total response length. The hyperparameter α balances the two loss terms.

Iterative Preference Optimization allows the use of negative examples for feedback, while methods like STaR [Zelikman et al., 2022] or ReST [Gulcehre et al., 2023] only use SFT on the generator and only include positive examples. [Wu, Lan, et al., 2024] follow using iterative DPO to train the base model with the reasoning chains, but without the NLL loss.

An important note for this paper: the NLL target c^w in IRPO is the *model's own winning chain-of-thought* for solving the task — not an external verifier's critique of the preference comparison between y_w and y_l. Method 1 of this work can be understood as IRPO where the NLL target is replaced with the oracle's preference rationale r.

3.3.3 AIPO: Improving Training Objective for Iterative Preference Optimization

AIPO [Shen et al., 2024] use a modified version of DPO and SFT:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta(s_\theta - s_{\text{ref}})\right)\right]$$

where

$$s_\theta = \log\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)}, \quad s_{\text{ref}} = \log\frac{\pi_{\text{ref}}(y_w|x)}{\pi_{\text{ref}}(y_l|x)}.$$

We note that s_{ref} represents the extent to which the reference model tends to generate the chosen response y_w with a higher probability than the rejected response y_l, i.e., the agreement between the reference model and the reward model. We introduce an additional coefficient α to s_{ref}:

$$\mathcal{L}_{\alpha\text{-DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta(s_\theta - (1+\alpha)s_{\text{ref}})\right)\right]$$

where we set $\alpha > 0$. The final loss is then:

$$\mathcal{L}_{\text{AIPO}} = \mathcal{L}_{\alpha\text{-DPO}} + \lambda \cdot \mathcal{L}_{\text{NLL}}$$

where $\mathcal{L}_{\text{NLL}} = -\frac{1}{|y_w|}\log(\pi_\theta(y_w|x))$.

Code available here

3.3.4 Regularized Preference Optimization

[Z. Liu et al., 2024] prove that the SFT loss on preferred responses acts as an implicit adversarial regularizer in RLHF, and use this insight to derive a principled combination of DPO and SFT losses that provably mitigates overoptimization of the reward model.

Code Repo available here. They use `OpenRLHF` for training and the `UltraFeedback` dataset.

3.3.5 Step-Level Preference Optimization

Standard DPO assigns a single preference signal to a complete response. A growing line of work argues that CoT reasoning requires *per-step* supervision because errors early in a chain propagate forward and cannot be corrected by an outcome-level signal.

[Lai et al., 2024] construct 10K step-wise preference pairs using self-generated data and apply DPO at the step level, arguing that whole-response comparison provides insufficient localisation of errors in long-chain mathematical reasoning. Applied to Qwen2-72B-Instruct, Step-DPO achieves 70.8% on MATH, surpassing GPT-4 at the time of publication.

3.4 Generative Verifiers and Critique for Reward Modeling

A parallel line of work focuses not on the policy model that generates answers, but on the verifiers and reward models that judge them. The key insight is that reward models which *generate* critique reasoning before scoring are more accurate and data-efficient than those which predict a scalar directly.

3.4.1 Generative Verifiers: Reward Modeling as Next-Token Prediction

The prevalent approach for training verifiers in reasoning domains is to fine-tune an LLM as a classifier on a dataset of correct and incorrect solutions generated from a fixed LLM, using binary cross-entropy loss. These verifiers directly assign a numerical score $r_\theta(x, y) \in [0, 1]$ to estimate the probability that a solution y is correct for a problem x. Discriminative LLM-based verifiers do not utilize the text generation capabilities of pretrained LLMs. To address this issue, [L. Zhang et al., 2024] propose training generative verifiers, which we call GenRM, using standard next-token prediction. GenRM represents solution correctness using the LLM's probability distribution over tokens, instead of predicting a separate numerical score.

Direct Verifier In its simplest form, GenRM predicts whether a solution is correct using a single 'Yes' or 'No' token. This can be done by maximizing $\log p_\theta(\text{'Yes'}|(x, y^+))$ for correct solutions y^+ and $\log p_\theta(\text{'No'}|(x, y^-))$ for incorrect solutions y^-. To do so, we minimize the SFT loss on the dataset $\mathcal{D}_{\text{Direct}}$ containing problem-solution pairs and a 'Yes' or 'No' verification token:

$$\mathcal{D}_{\text{Direct}} = \{(x, y^+, I), \text{'Yes'}\} \cup \{(x, y^-, I), \text{'No'}\} \tag{6}$$

where $I =$ 'Is the answer correct (Yes/No)?'. At inference, we use the likelihood of the 'Yes' token as the verifier's score for re-ranking solutions.

Unifying Generation and Verification GenRM seamlessly integrates reward modeling with SFT for generating correct solutions. Given a verification dataset $\mathcal{D}_{\text{verify}}$, GenRM minimizes the loss:

$$\mathcal{L}_{\text{GenRM}}(\theta, \mathcal{D}_{\text{verify}}) = \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{verify}}) + \lambda \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{correct}}) \tag{7}$$

where $\lambda > 0$ controls the mixture ratio between verification and generating correct solutions.

Chain-of-Thought Verifiers (GenRM-CoT) Since verification often involves nuanced reasoning, generative verifiers can naturally benefit from Chain-of-Thought. We can generate intermediate reasoning steps before making a decision about correctness. To train CoT verifiers, we minimize the SFT loss on dataset $\mathcal{D}_{\text{CoT}}$ containing problem-solution pairs and corresponding verification rationales v_{CoT}:

$$\mathcal{D}_{\text{CoT}} = \{(x, y^+, I_{\text{CoT}}), (v_{\text{CoT}}, I, \text{'Yes'})\} \cup \{(x, y^-, I_{\text{CoT}}), (v_{\text{CoT}}, I, \text{'No'})\} \tag{8}$$

where I_{CoT} = 'Let's verify step by step.'. During inference, we first generate a CoT rationale and then use the probability of 'Yes' for assigning the correctness score. The CoT verification rationale v_{CoT} is precisely the oracle reasoning r that Method 1 of this paper uses as an NLL training target for the base policy model.

Data for the CoT verification rationales v_{CoT} is available here

3.4.2 Improving Reward Models with Synthetic Critiques

[Zihuiwen Ye et al., 2024] use externally generated critique of the preference dataset to train reward models jointly with those critiques. Once we have the critiques c of the preference dataset generated from an external LLM, we augment the training data by enriching them with critiques. We achieve this by concatenating the critiques after each completion to form new preference pairs $(x, y_+ : c_+, y_- : c_-)$, which become a critique-augmented training set $\mathcal{D}'$. We train critique RMs on $\mathcal{D}'$ with the loss:

$$\mathcal{L}(\theta, \mathcal{D}') = -\mathbb{E}_{(x,y_+,y_-,c)\sim\mathcal{D}'} \log(\sigma(r_\theta(x, y_+ : c_+) - r_\theta(x, y_- : c_-) - m(r))) \tag{9}$$

We implement our RMs by replacing the final projection layer of a pretrained language model with a linear layer that predicts a logit. This is similar to discriminative verifiers.

3.4.3 Critique-Out-Loud Reward Models

CLoud reward models [Ankner et al., 2024] are trained with a dataset of N examples, $D = \{(x, y^-, y^+, c^-, c^+)_i\}_{i=1}^N$ where we introduce oracle critiques, c^-, c^+, of the rejected and chosen responses y^-, y^+ respectively. The critiques are reasoning traces that provide feedback on the weaknesses of the responses and strategies for improving them.

To train CLoud reward models we: (1) train the base model and LM head to generate critiques via supervised finetuning on the oracle critiques, (2) replace oracle critiques in the dataset with critiques generated by the finetuned model, and (3) train a reward head conditioned on self-generated critiques. We choose to train the reward head on self-generated critiques so as to minimise the distribution shift in the critiques seen by the reward head between training and inference when oracle critiques are not available.

First, we modify L_{RM} to work with CLoud reward models as:

$$L_{RM}(\theta_B, \theta_R, D) = -\mathbb{E}_{(x,y^-,y^+,c^-,c^+)\sim D}[\log(\sigma(r_{\theta_B,\theta_R}(x, y^+, c^+) - r_{\theta_B,\theta_R}(x, y^-, c^-)))]$$

where the reward estimator $r_\theta(x, y, c)$ is now also conditioned on a critique c. Next, we introduce the critique SFT loss, which is the negative log likelihood of the rejected and chosen critiques:

$$L_{SFT}(\theta_B, \theta_{LM}, D) = -\mathbb{E}_{(x,y^-,y^+,c^-,c^+)\sim D}\left[\sum_{c_t^- \in c^-} \log p(c_t^- | x, y^-, c_{<t}^-; \theta_B, \theta_{LM}) + \sum_{c_t^+ \in c^+} \log p(c_t^+ | x, y^+, c_{<t}^+; \theta_B, \theta_{LM})\right]$$

where $c_{<t} = c_1, ..., c_{t-1}$ is the prefix of the critique up to the tth token. Finally, we introduce a joint SFT and RM loss as:

$$L_{CLoud}(\theta_B, \theta_{LM}, \theta_R, D) = L_{RM}(\theta_B, \theta_R, D) + \lambda \cdot L_{SFT}(\theta_B, \theta_{LM}, D)$$

where λ is a hyperparameter that weights the contribution of the language modeling loss.

To train CLoud reward models we first train θ_B, θ_{LM} to generate critiques by minimizing L_{SFT} on the oracle critiques in the oracle dataset D and obtain parameters $\tilde{\theta}_B, \tilde{\theta}_{LM}$. We use the finetuned model

to modify our dataset with self-generated critiques that replace the oracle critiques. Finally, we obtain our CLoud reward model parameters by training on our self-critique dataset:

$$\theta_B^*, \theta_{LM}^*, \theta_R^* = \arg\min_{\theta_B, \theta_{LM}, \theta_R} L_{CLoud}(\theta_B, \theta_{LM}, \theta_R, D_{self})$$

where θ_B, θ_{LM} are initialized to the finetuned parameters $\tilde{\theta}_B, \tilde{\theta}_{LM}$. We train CLoud reward models on the joint loss L_{CLoud} instead of L_{RM} to preserve the critique generation capability of the CLoud reward model and to prevent over-fitting to solely producing reward scores.

Code and Data Code and data available here.

3.4.4 Self-Generated Critiques Boost Reward Modeling

Recent work [Zihuiwen Ye et al., 2024] directly incorporates critiques generated from off-the-shelf LLMs for reward modeling, while [Ankner et al., 2024] and [L. Zhang et al., 2024] design a joint training approach for learning to generate the critique as well as rewards simultaneously via knowledge distillation. These methods typically rely on a strong teacher LLM to generate high-quality critiques, which can be costly and inefficient to obtain at scale in practice. Moreover, they cannot be used to improve frontier models when a stronger teacher model does not exist.

Critic-RM [Yu et al., 2025] is a framework to allow LLMs to take advantage of self-generated critiques for reward modeling. Critic-RM first generates candidate critiques for each prompt-response pair. Then, a filtering step is conducted to reduce the effect of potentially noisy rationales leading to incorrect predictions, allowing augmentation of the preference pairs with additional critiques. Finally, we implement a joint training scheme to teach the model both high-quality critique generation and accurate reward modeling.

Critique-augmented Reward Model Training We denote z^+, z^- as critiques for chosen and rejected responses y^+, y^- with prompt x, respectively. Then, the overall learning objective $p(y^+ \succ y^-|x)$ can be recast as:

$$\mathcal{L} = \mathbb{E}_{q_\phi(z^+|y^+,x), q_\phi(z^-|y^-,x)} \left[-\log \frac{p(y^+ \succ y^-, z^+, z^-|x)}{q_\phi(z^+|y^+,x) q_\phi(z^-|y^-,x)} \right]$$

We decompose the reward model training objective into 2 parts:

1) Preference Modeling Loss with Critiques ℓ_r: the reward model r_θ learns to predict the reward for each response conditioned on critiques

$$\ell_r(x, y^+, y^-, z^+, z^-) = -\log p(y^+ \succ y^-, z^+, z^-|x) = -\log p(r_\psi(x, [y^+; z^+]) > r_\psi(x, [y^-; z^-]))$$

2) Critique Generation Loss ℓ_c: the LLM generation g_θ is trained to generate critiques to approximate the oracle distribution $p^*(\cdot|y, x)$.

$$\ell_c(Z; x, y) = D_{KL}(p^*(z|y_i, x_i)||q_\phi(z|y_i, x_i)) = -\frac{1}{K} \sum_{z \in Z} \log q_\phi(z|y, x) + \text{const.}$$

The overall loss for critique generation is $\ell_c(x, y^+, y^-, Z^+, Z^-) = \ell_c(Z^+; x, y^+) + \ell_c(Z^-; x, y^-)$. We then jointly train the reward model r_ψ and critique generation model g_ϕ.

3.4.5 Generative Reward Models as Judges

[Mahan et al., 2024] propose a framework where an LLM π_ϕ acts as a zero-shot judge in an RLAIF setting. Given a task x and two responses y_1 and y_2, the model provides an answer indicator token I indicating a preference over the answers. They consider two variants:

1. The Generative Reward Model (GenRM) approach prompts the model to act as a classifier directly providing the answer token probabilities for each response $\hat{I} \sim \pi_\phi(I|x, y_1, y_2)$.

2. The CoT-GenRM approach additionally prompts the model to provide intermediate Chain-of-Thought reasoning $\hat{I}, \hat{r} \sim \pi_\phi(I, r|x, y_1, y_2)$ before providing the final answer indicator token.

GenRM (no CoT) To train the GenRM model, we use the standard supervised fine tuning objective

$$\mathcal{L}_{\text{GenRM}}(\pi_\phi) = \mathbb{E}_{(x,y_1,y_2,I)\sim\mathcal{D}}[-\log \pi_\phi(I|x, y_1, y_2)] \tag{10}$$

essentially using the LLM as a classifier trained with next-token prediction.

CoT-GenRM with Rationalization To train the CoT-GenRM we bootstrap intermediate reasoning from a rationalization model $r \sim q_\phi(r|x^{(i)}, y_1^{(i)}, y_2^{(i)})$ and train with maximum likelihood over both the reasoning chain and ranking:

$$\mathcal{L}_{\text{GenRM-Rationalization}}(\pi_\phi) = \mathbb{E}_{(x,y_1,y_2,r,I)\sim\mathcal{D}}[-\log \pi_\phi(I|x, y_1, y_2, r) - \log \pi_\phi(r|x, y_1, y_2)] \tag{11}$$

CoT-GenRM-STaR Finally we consider an approach where the model self-bootstraps intermediate reasoning using a STaR approach. We create a dataset of preference pairs $\mathcal{D} = \{x^{(i)}, y_1^{(i)}, y_2^{(i)}, r_w^{(i)}, I_w^{(i)}, r_l^{(i)}, I_l^{(i)}\}$, where r_w are rationales that lead to correct rankings I_w and r_l are rationales that lead to incorrect rankings I_l. We then use a DPO optimization objective of the form:

$$\mathcal{L}_{\text{GenRM-DPO}}(\pi_\phi) = \mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\phi(I_w, r_w|x, y_1, y_2)}{\pi_{\text{ref}}(I_w, r_w|x, y_1, y_2)} - \beta \log \frac{\pi_\phi(I_l, r_l|x, y_1, y_2)}{\pi_{\text{ref}}(I_l, r_l|x, y_1, y_2)}\right)\right] \tag{12}$$

We denote models trained with this objective as STaR-DPO. Note that this is equivalent in form to Method 1 of this paper, but applied to train a standalone *judge* model rather than the base policy model that generates answers.

Direct Judgement Preference Optimization (DJPO) [P. Wang et al., 2024] apply the same DPO + NLL objective structure as equation 5 to train a dedicated judge model. They collect preference pairs through three approaches (human annotation, self-generation, and retrieval) and use the verifier's reasoning trace as the NLL target. The resulting judge model learns both to rank responses and to articulate *why* one is preferred. Structurally, DJPO has the same loss form as Method 1 of this paper — DPO on preference labels combined with NLL on the verifier's rationale — with the critical distinction that it trains a standalone judge model (not the base policy model that generates the original answers).

Self-Taught Evaluators [T. Wang et al., 2024] curate a preference dataset (x_i, y_i^w, y_i^l) and use a seed LLM to generate reasoning traces c_i on ranking the outputs. They then iteratively fine-tune the evaluator LLM on the next iteration of reasoning traces (x_i, y_i^w, y_i^l, c_i), to create a better evaluator. [Ziyi Ye et al., 2024] similarly use an LLM to rank preferences and use the rankings in DPO, and the CoT data in SFT, to train a generative judge.

Meta-Rewarding [Wu, W. Yuan, et al., 2024] uses the model to judge its own judgements and uses that feedback to refine its judgment skills, creating an iterative loop that improves both generation and evaluation quality simultaneously.

A recent trend extends this line of work further, training *thinking* LLM judges that reason at length before issuing a preference score. Empirical evidence for the value of this approach comes from Kimi k1.5 [Kimi Team, 2025], which reports that their Chain-of-Thought reward model achieves 98.5% annotation accuracy versus 84.4% for a classical scalar reward model on a challenging preference dataset — a 14-point gap that provides direct quantitative motivation for using verifier reasoning as a training signal.

3.4.6 Process Reward Models

Process Reward Models (PRMs) assign a reward to *each* step of a reasoning chain rather than to the final answer alone.

Let's Verify Step by Step (PRM800K) [Lightman et al., 2024] demonstrate that process supervision substantially outperforms outcome supervision on MATH, achieving 78% accuracy with a process-supervised reward model compared to 72.4% for outcome supervision. The paper releases PRM800K, a dataset of 800K step-level human annotations, and remains the canonical reference for step-level feedback in reasoning. The high cost of collecting PRM800K motivates subsequent work on automated and generative PRMs.

ThinkPRM: Process Reward Models That Think [Khalifa et al., 2025] is perhaps the closest existing work to the Critique DPO generative-verifier idea at the process level. ThinkPRM generates a verification chain-of-thought *before* scoring each reasoning step rather than predicting a scalar. Using only 1% of the PRM800K labels, ThinkPRM surpasses discriminative PRMs trained on the full dataset by 8% on GPQA-Diamond, demonstrating that generative CoT verifiers are substantially more data-efficient than scalar counterparts. This result validates the core premise of this paper: that verifier reasoning is not merely a byproduct to be discarded, but a signal of genuine value that, when used during training, yields substantial performance gains.

[Zhao et al., 2025] extend the generative PRM idea by combining natural-language reasoning with executable code verification before assigning a step score. A 7B GenPRM exceeds Qwen2.5-Math-PRM-72B using only 23K training examples, further establishing that generative verifier reasoning provides a strong inductive bias for process supervision.

3.4.7 Outcome-Process Inconsistency: RM-NLHF

[Z. Wang et al., 2026] identify a fundamental failure mode of training Generative Reward Models (GRMs) with outcome-only RL objectives: *outcome-process inconsistency*, where a GRM correctly predicts the preferred response but generates a logically flawed or hallucinated rationale to justify that prediction. Because standard RLVR objectives penalise only incorrect outcome labels, GRMs can exploit spurious correlations in the binary preference classification without developing genuine evaluative reasoning. Quantifying this across frontier models, the authors find that 20–33% of correct GRM predictions are accompanied by unsound reasoning chains — including Gemini-2.5-Pro (26%) and Claude-3.7-Sonnet (33.6%).

To address this, RM-NLHF adds a *process reward* term R_{process} to the training signal, set to 1 when the F1 similarity between the GRM's generated critique and a human oracle critique exceeds a threshold, and 0 otherwise. The composite training reward is:

$$r(x, y^+, y^-) = \begin{cases} -1 & \text{invalid format} \\ 0 & \text{incorrect outcome label} \\ 1 + \lambda \cdot R_{\text{process}} & \text{correct outcome label} \end{cases} \tag{13}$$

Since collecting human critiques at scale is expensive, a MetaRM — a small regression model trained on a seed human-critique dataset — predicts R_{process} for unlabelled data, scaling the approach cost-effectively. An online variant of MetaRM co-evolves with GRM training to handle distribution shift.

Results demonstrate that RM-NLHF-trained models substantially outperform outcome-only baselines: RM-NLHF-Qwen-7B achieves an overall score of 0.648 versus 0.576 for RM-R1-Qwen-7B. When used as the reward model in a downstream RL pipeline, RM-NLHF improves policy performance by 12.9% on Arena-Hard-V2.0. This paper provides perhaps the strongest empirical argument for the core premise of Critique DPO: that supervising the *quality* of verifier reasoning — not just the correctness of its outcome label — is essential for robust alignment.

3.5 Training the Base Model with Critique

The methods reviewed in Section 3.4 improve reward models and judges by having them generate critique reasoning. A distinct and closely related cluster trains the base *policy* model directly using critique signals — either through an NLL objective on annotated critiques, RL with critique-structured rewards, or distillation from a teacher's correction traces. This section covers the methods most directly related to Method 1 of this paper.

The three defining features of Method 1 are: (A) a DPO loss on answer preferences (y_w, y_l); (B) an NLL term on the oracle/verifier's CoT rationale r (explaining why $y_w \succ y_l$); and (C) applied to the base policy model. No single prior paper combines all three; the methods below each match two of the three.

3.5.1 Critique Fine-Tuning (CFT)

[Y. Wang et al., 2025] CFT involves training the model to critique a given query-response pair, maximizing the likelihood $P(c|[x; y])$, where c is the annotated critique for a query-response pair $[x; y]$. CFT yields

a consistent 4–10% improvement over SFT on six math benchmarks. Notably, their model Qwen2.5-Math-CFT only requires 1 hour of training on 8×H100 over 50K examples. It can match or outperform strong competitors like Qwen2.5-Math-Instruct on most benchmarks, which use over 2M samples.

CFT matches features (B) and (C) of Method 1 exactly: it applies NLL on critique reasoning to the base model. The missing component is feature (A) — the DPO preference term. Method 1 can be understood as CFT augmented with a DPO preference term that simultaneously encourages the model to prefer better answers.

3.5.2 CriticGPT

[McAleese et al., 2024] train an LLM critique model via RLHF on human-annotated errors injected into ChatGPT outputs. The key finding is that CriticGPT-trained critics are preferred over unassisted human critics in 63% of cases and catch approximately 85% of injected bugs, compared to approximately 25% for humans alone. The paper validates the scalable-oversight motivation for critique-based training: LLMs can be explicitly taught to generate high-quality, targeted critiques that outperform human reviewers in detecting fine-grained errors, providing a practical recipe for constructing the oracle critique data r used in Method 1.

3.5.3 CTRL: Teaching Critique via Reinforcement Learning

[Xie et al., 2025] (CTRL) train a critic model with RL to maximise correction performance for a fixed generator, without requiring human supervision. The approach produces critiques that serve as accurate generative reward signals and enables test-time scaling through iterative critique–revision cycles, with up to 106% relative improvement on challenging code-generation benchmarks. CTRL demonstrates that high-quality critiques can be learned via RL from outcome signals alone, without oracle critique data — a property relevant to settings where constructing the oracle rationale r for Method 1 is expensive.

3.5.4 SuperCorrect

[Yang et al., 2024] introduce a two-stage framework: (1) hierarchical thought template distillation from a large teacher model, and (2) cross-model collaborative DPO training on teacher-guided error-correction traces. SuperCorrect-7B surpasses DeepSeekMath-7B by 7.8%/5.3% on MATH/GSM8K. The teacher's error-correction trace plays the role of r in Method 1 — an external model's reasoning about what went wrong and how to fix it — but the correction is embedded inside the DPO preference pair structure rather than as a separate NLL term, making it a close structural cousin of Method 1.

3.5.5 Critique-GRPO

[X. Zhang et al., 2025] is the closest existing paper to Method 1's core proposal. Critique-GRPO integrates natural-language critiques and numerical rewards jointly in a GRPO-based RL framework, training the base policy model to learn from both initial responses and critique-guided self-refinements simultaneously. The framework introduces a specialized shaping function that amplifies learning from "unfamiliar" critique-guided refinements — trajectories the model successfully navigated using the critique but would not have explored autonomously. Critique-GRPO achieves +4.4% on Qwen2.5-7B-Base and +16.7% pass@1 on AIME 2024.

The key differences from Method 1 are:

	Method 1	**Critique-GRPO**
Algorithm	DPO (offline)	GRPO (online RL)
Critique source	External oracle	Self-generated
Training regime	Offline preference pairs	Online rollouts

These two papers arrive at the same conceptual combination — critique reasoning and preference optimisation for the base model — from different training paradigms.

3.5.6 iGRPO: Self-Feedback-Driven Reasoning

[Hatamizadeh et al., 2026] extend GRPO with a two-stage self-feedback loop that directly outperforms Critique-GRPO without requiring explicit natural-language critique generation. In Stage 1, N candidate responses are sampled from the current policy and scored with a rule-based verifier; the highest-scoring draft $d_\theta(q)$ is selected. In Stage 2, the original prompt is augmented with this best draft and G new completions are sampled from the conditioned policy and used for standard GRPO updates. The total sampling budget is held constant ($N + G = G_{\text{GRPO}}$) for a fair comparison.

The authors prove that the expected reward of the selected Stage 1 draft is $1 - (1 - V(q))^N$, which is monotonically increasing in the policy's success probability $V(q)$: as the policy improves, the conditioning signal improves, creating a bootstrapping dynamic. iGRPO also delays entropy collapse during training, maintaining broader exploration of reasoning pathways.

Results: Nemotron-H-8B-Base (GRPO 41.1% → Critique-GRPO 43.4% → iGRPO **45.0%**); DeepSeek-R1-Distill-Qwen-7B (Critique-GRPO 69.1% → iGRPO **69.9%**); state-of-the-art for 7B-class models: AIME 2024 **85.62%**, AIME 2025 **79.64%** (OpenReasoning-Nemotron-7B).

These results suggest that self-generated high-quality drafts can serve as an effective implicit critique signal without verbal critique generation, and provide an important point of comparison for the offline DPO approach of Method 1.

3.5.7 RL Tango: Joint Generator–Verifier Training

[Zha et al., 2025] simultaneously train both a policy generator and a generative process-level verifier using interleaved RL, with the verifier rewarded solely on outcome correctness without explicit process labels. The co-evolving pair achieves best-in-class performance on competition mathematics while the verifier leads ProcessBench, providing practical evidence that the joint generator–verifier training loop implicit in Critique DPO can work without pre-labelled critique data.

3.6 Step-Level and Process Critique

[S. Yuan et al., 2025] uses step-level model guided critique and MCTS to iteratively refine the model. [D. Zhang et al., 2024] follow a similar approach. [Xi et al., 2024] create AutoMathCritique — an automated and scalable framework for collecting critique data. This framework consists of three main stages: flawed reasoning path construction, critique generation, and data filtering. [Paul et al., 2024] use critique to refine the model's intermediate steps.

References

Rafailov, Rafael et al. (2024). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. arXiv: 2305.18290 [cs.LG]. URL: https://arxiv.org/abs/2305.18290.

Wolfe, Cameron R. (2025). *Direct Preference Optimization (DPO)*. Blog post. URL: https://cameronrwolfe.substack.com/p/direct-preference-optimization.

Romero, Tyler (2024). *Direct Preference Optimization Explained In-depth*. Blog post. URL: https://www.tylerromero.com/posts/2024-04-dpo/.

Bai, Yuntao et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv: 2212.08073 [cs.CL]. URL: https://arxiv.org/abs/2212.08073.

Lee, Harrison et al. (2024). *RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*. arXiv: 2309.00267 [cs.CL]. URL: https://arxiv.org/abs/2309.00267.

Guo, Shangmin et al. (2024). *Direct Language Model Alignment from Online AI Feedback*. arXiv: 2402.04792 [cs.AI]. URL: https://arxiv.org/abs/2402.04792.

Li, Ang et al. (2024). *HRLAIF: Improvements in Helpfulness and Harmlessness in Open-domain Reinforcement Learning From AI Feedback*. arXiv: 2403.08309 [cs.LG]. URL: https://arxiv.org/abs/2403.08309.

Zelikman, Eric et al. (2022). *STaR: Bootstrapping Reasoning With Reasoning*. arXiv: 2203.14465 [cs.LG]. URL: https://arxiv.org/abs/2203.14465.

Hosseini, Arian et al. (2024). *V-STaR: Training Verifiers for Self-Taught Reasoners*. arXiv: 2402.06457 [cs.LG]. URL: https://arxiv.org/abs/2402.06457.

Liu, Hanbing et al. (2025). *Uncovering the Impact of Chain-of-Thought Reasoning for Direct Preference Optimization*. arXiv: 2502.11656 [cs.CL]. URL: https://arxiv.org/abs/2502.11656.
Yuan, Weizhe et al. (2024). *Self-Rewarding Language Models*. arXiv: 2401.10020 [cs.CL]. URL: https://arxiv.org/abs/2401.10020.
Xu, Jing et al. (2024). *Some things are more CRINGE than others: Iterative Preference Optimization with the Pairwise Cringe Loss*. arXiv: 2312.16682 [cs.CL]. URL: https://arxiv.org/abs/2312.16682.
Pang, Richard Yuanzhe et al. (2024). *Iterative Reasoning Preference Optimization*. arXiv: 2404.19733 [cs.CL]. URL: https://arxiv.org/abs/2404.19733.
Gulcehre, Caglar et al. (2023). *Reinforced Self-Training (ReST) for Language Modeling*. arXiv: 2308.08998 [cs.CL]. URL: https://arxiv.org/abs/2308.08998.
Wu, Tianhao, Janice Lan, et al. (2024). *Thinking LLMs: General Instruction Following with Thought Generation*. arXiv: 2410.10630 [cs.CL]. URL: https://arxiv.org/abs/2410.10630.
Shen, Yaojie et al. (2024). *AIPO: Improving Training Objective for Iterative Preference Optimization*. arXiv: 2409.08845 [cs.CL]. URL: https://arxiv.org/abs/2409.08845.
Liu, Zhihan et al. (2024). *Provably Mitigating Overoptimization in RLHF: Your SFT Loss is Implicitly an Adversarial Regularizer*. arXiv: 2405.16436 [cs.LG]. URL: https://arxiv.org/abs/2405.16436.
Lai, Xin et al. (2024). *Step-DPO: Step-wise Preference Optimization for Long-chain Reasoning of LLMs*. arXiv: 2406.18629 [cs.LG]. URL: https://arxiv.org/abs/2406.18629.
Zhang, Lunjun et al. (2024). *Generative Verifiers: Reward Modeling as Next-Token Prediction*. arXiv: 2408.15240 [cs.LG]. URL: https://arxiv.org/abs/2408.15240.
Ye, Zihuiwen et al. (2024). *Improving Reward Models with Synthetic Critiques*. arXiv: 2405.20850 [cs.CL]. URL: https://arxiv.org/abs/2405.20850.
Ankner, Zachary et al. (2024). *Critique-out-Loud Reward Models*. arXiv: 2408.11791 [cs.LG]. URL: https://arxiv.org/abs/2408.11791.
Yu, Yue et al. (2025). *Self-Generated Critiques Boost Reward Modeling for Language Models*. arXiv: 2411.16646 [cs.CL]. URL: https://arxiv.org/abs/2411.16646.
Mahan, Dakota et al. (2024). *Generative Reward Models*. arXiv: 2410.12832 [cs.LG]. URL: https://arxiv.org/abs/2410.12832.
Wang, Peifeng et al. (2024). *Direct Judgement Preference Optimization*. arXiv: 2409.14664 [cs.CL]. URL: https://arxiv.org/abs/2409.14664.
Wang, Tianlu et al. (2024). *Self-Taught Evaluators*. arXiv: 2408.02666 [cs.CL]. URL: https://arxiv.org/abs/2408.02666.
Ye, Ziyi et al. (2024). *Beyond Scalar Reward Model: Learning Generative Judge from Preference Data*. arXiv: 2410.03742 [cs.CL]. URL: https://arxiv.org/abs/2410.03742.
Wu, Tianhao, Weizhe Yuan, et al. (2024). *Meta-Rewarding Language Models: Self-Improving Alignment with LLM-as-a-Meta-Judge*. arXiv: 2407.19594 [cs.CL]. URL: https://arxiv.org/abs/2407.19594.
Kimi Team (2025). *Kimi k1.5: Scaling Reinforcement Learning with LLMs*. arXiv: 2501.12599 [cs.CL]. URL: https://arxiv.org/abs/2501.12599.
Lightman, Hunter et al. (2024). "Let's Verify Step by Step". In: *ICLR*. arXiv: 2305.20050 [cs.LG]. URL: https://arxiv.org/abs/2305.20050.
Khalifa, Muhammad et al. (2025). *Process Reward Models That Think*. arXiv: 2504.16828 [cs.LG]. URL: https://arxiv.org/abs/2504.16828.
Zhao, Jian et al. (2025). *GenPRM: Scaling Test-Time Compute of Process Reward Models via Generative Reasoning*. arXiv: 2504.00891 [cs.LG]. URL: https://arxiv.org/abs/2504.00891.
Wang, Zongqi et al. (2026). *Reward Modeling from Natural Language Human Feedback*. arXiv: 2601.07349 [cs.CL]. URL: https://arxiv.org/abs/2601.07349.
Wang, Yubo, Xiang Yue, and Wenhu Chen (2025). *Critique Fine-Tuning: Learning to Critique is More Effective than Learning to Imitate*. arXiv: 2501.17703 [cs.CL]. URL: https://arxiv.org/abs/2501.17703.
McAleese, Nat et al. (2024). *LLM Critics Help Catch LLM Bugs*. arXiv: 2407.00215 [cs.CL]. URL: https://arxiv.org/abs/2407.00215.
Xie, Zhihui et al. (2025). *Teaching Language Models to Critique via Reinforcement Learning*. arXiv: 2502.03492 [cs.CL]. URL: https://arxiv.org/abs/2502.03492.

Yang, Ling et al. (2024). *SuperCorrect: Advancing Small LLM Reasoning with Thought Template Distillation and Self-Correction*. arXiv: 2410.09008 [cs.CL]. URL: https://arxiv.org/abs/2410.09008.

Zhang, Xiaoying et al. (2025). *Critique-GRPO: Advancing LLM Reasoning with Natural Language and Numerical Feedback*. arXiv: 2506.03106 [cs.CL]. URL: https://arxiv.org/abs/2506.03106.

Hatamizadeh, Ali et al. (2026). *iGRPO: Self-Feedback-Driven LLM Reasoning*. arXiv: 2602.09000 [cs.CL]. URL: https://arxiv.org/abs/2602.09000.

Zha, Kaiwen et al. (2025). *RL Tango: Reinforcing Generator and Verifier Together for Language Reasoning*. arXiv: 2505.15034 [cs.CL]. URL: https://arxiv.org/abs/2505.15034.

Yuan, Siyu et al. (2025). *Agent-R: Training Language Model Agents to Reflect via Iterative Self-Training*. arXiv: 2501.11425 [cs.AI]. URL: https://arxiv.org/abs/2501.11425.

Zhang, Di et al. (2024). *LLaMA-Berry: Pairwise Optimization for O1-like Olympiad-Level Mathematical Reasoning*. arXiv: 2410.02884 [cs.AI]. URL: https://arxiv.org/abs/2410.02884.

Xi, Zhiheng et al. (2024). *Enhancing LLM Reasoning via Critique Models with Test-Time and Training-Time Supervision*. arXiv: 2411.16579 [cs.CL]. URL: https://arxiv.org/abs/2411.16579.

Paul, Debjit et al. (2024). *REFINER: Reasoning Feedback on Intermediate Representations*. arXiv: 2304.01904 [cs.CL]. URL: https://arxiv.org/abs/2304.01904.